

Implementación de Lenguajes Orientados a Objetos

Introducción

Un **lenguaje orientado a objetos** debe cumplir con tres requisitos:

- Tener un mecanismo para definir tipos de datos abstractos.
- Permitir alguna forma de herencia.
- Implementar ligadura dinámica de código para lograr polimorfismo.

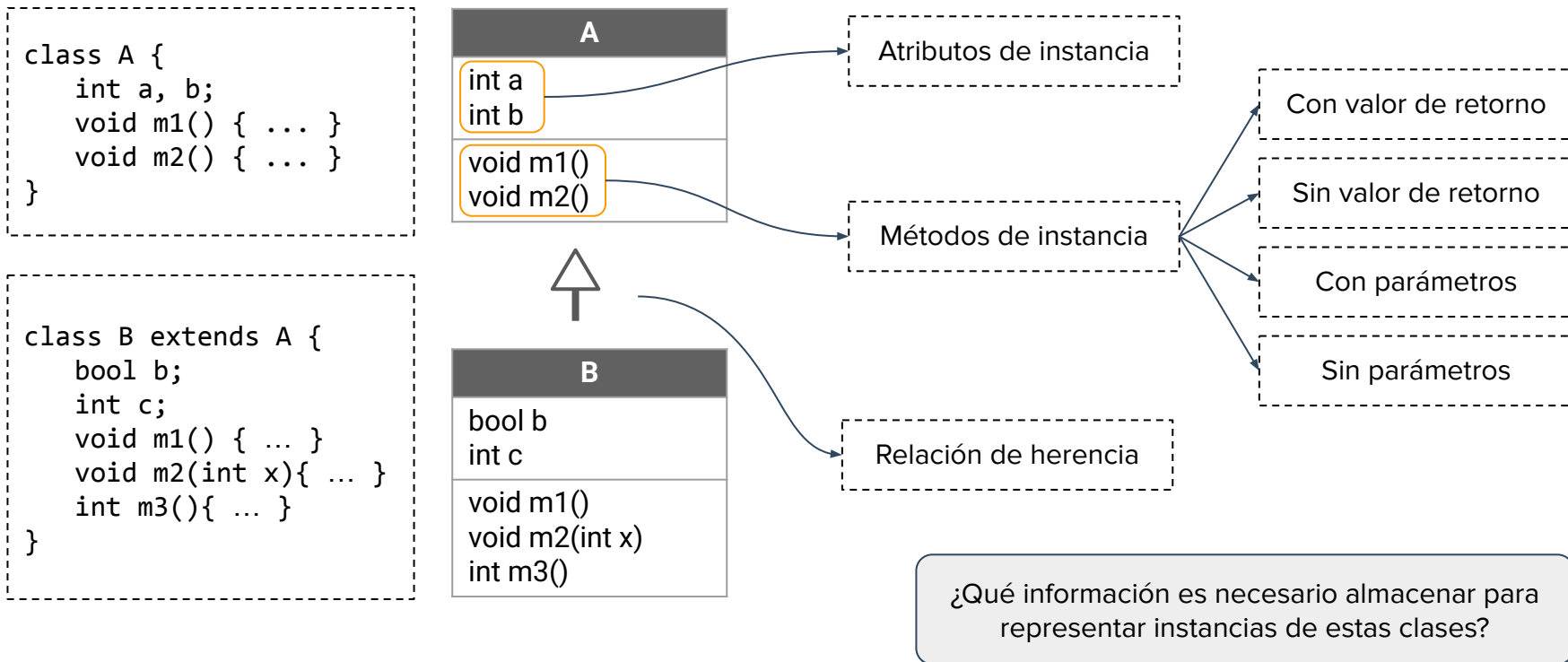
Desde el punto de vista de la implementación, estas características requieren cierta infraestructura en tiempo de ejecución.

Nos centraremos en un lenguaje OO con **tipado estático**, como lo es **Java**.

Veremos cómo las construcciones del lenguaje pueden ser ejecutadas siguiendo una **secuencia equivalente de instrucciones del *procesador abstracto SimpleSem***.

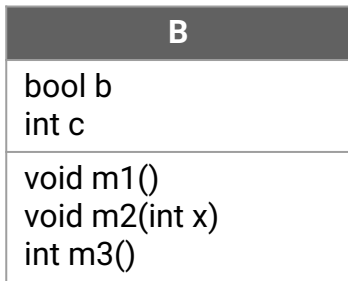
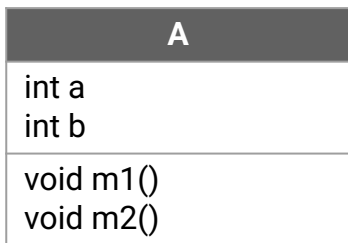
Introducción

Podemos considerar dos clases con distintos atributos y métodos.



Introducción

Podemos considerar dos clases con distintos atributos y métodos.

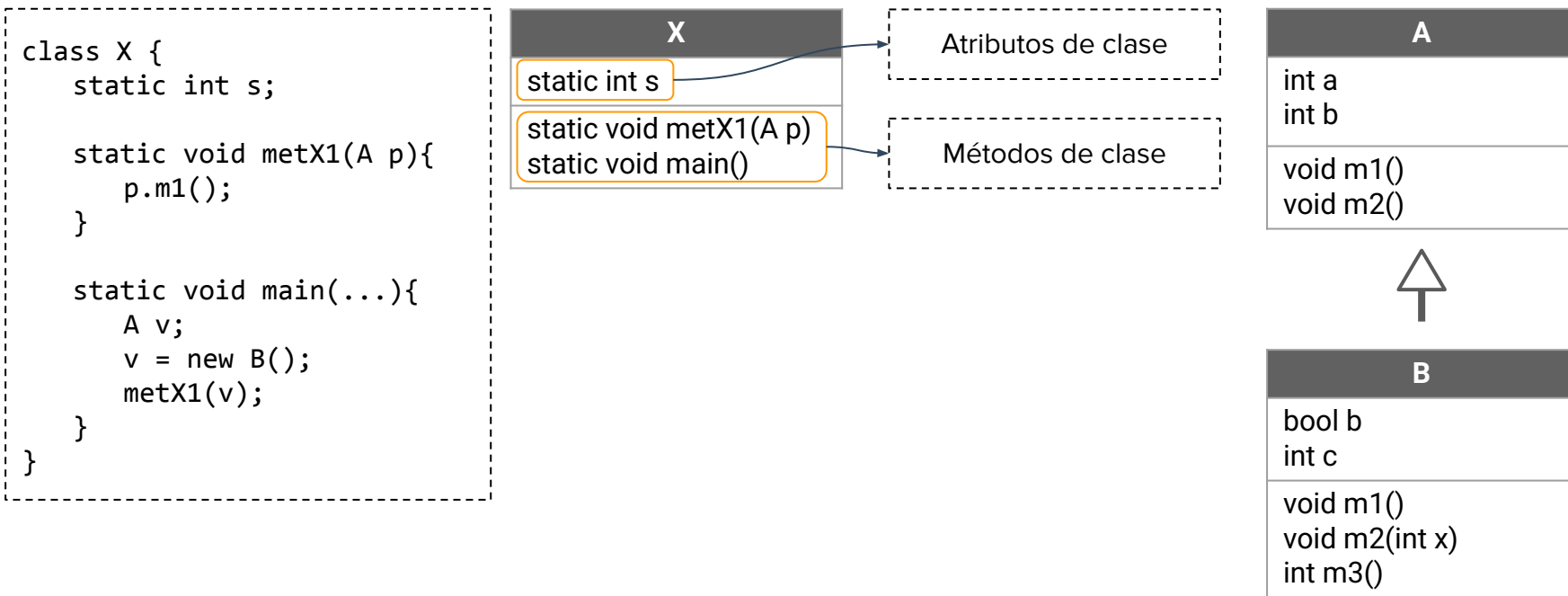


```
class A {  
    int a, b;  
  
    void m1() {  
        a = 1;  
        b = -1;  
    }  
  
    void m2() {  
        a = b + 1;  
        b = b + 5;  
    }  
}
```

```
class B extends A {  
    bool b;  
    int c;  
  
    void m1() {  
        super.m1();  
        c = m3();  
        b = true;  
    }  
    void m2(int x){  
        x = x + 1;  
        a = a + x;  
    }  
    int m3(){  
        m2(1);  
        return a;  
    }  
}
```

Introducción

Vamos a considerar una clase adicional que actuará como cliente.



Introducción

A lo largo de la presentación iremos viendo cómo se resuelven diversas cuestiones de implementación.

Traducción

¿Cómo hacemos la traducción de los métodos a un lenguaje como SimpleSem?

Manejo de memoria

¿Qué estructuras se necesitan en memoria para lograr la ejecución del programa?
¿Dónde se almacenan los objetos? ¿Y el código de los métodos?
¿Qué diferencia existe en la implementación de métodos de instancia o de clase?

Herencia

¿Cómo hereda atributos y métodos una clase derivada de su clase padre?
¿Qué sucede con los atributos y métodos redefinidos en la clase derivada?
¿Qué sucede cuando tenemos sobrecarga de métodos?

Ligadura dinámica de código

¿Cómo hace un objeto para responder de forma adecuada a un mensaje?

Procesador Abstracto SimpleSem

SimpleSem

Vamos a utilizar un lenguaje llamado SimpleSem para escribir programas.

```
public class A {  
    int x,y;  
  
    void met1(int p){  
        x = p;  
        y = p*5;  
    }  
  
    int met2(){  
        int rtn;  
        if((x+y)/2 > 10){  
            rtn = x+2;  
        }  
        else{  
            rtn = y*5;  
        }  
        return rtn  
    }  
}
```

%%%----- Traducción de met1 de la clase A -----

```
met1A    SetH D[Actual+2]+1, D[Actual+3]  
          SetH D[Actual+2]+2, D[Actual+3]*5  
          SetLibre Actual  
          SetActual D[Libre+1]  
          Jump D[Libre]
```

%%%----- Traducción de met2 de la clase A -----

```
met2A    JumpT PC+3, (H[D[Actual+2]+1]+H[D[Actual+2]+2])/2 > 10  
          SetD Actual+3, H[D[Actual+2]+2]*5  
          Jump PC+2  
          SetD Actual+3, H[D[Actual+2]+1]+2  
          SetD Actual-1, D[Actual+3]  
          SetLibre Actual  
          SetActual D[Libre+1]  
          Jump D[Libre]
```


Componentes del Procesador Abstracto

El procesador SimpleSem está compuesto por **tres unidades de memoria**:

Memoria de código (C)
Almacena las traducciones de los métodos del código fuente.
Memoria de datos (D)
Almacena datos estáticos (Registros de clase, Virtual Tables) y los registros de activación.
Memoria dinámica o Heap (H)
Almacena los Registros de Instancia de cada objeto creado en ejecución.

Utilizaremos **cuatro registros especiales** para manipular la memoria.

Funcionamiento del Procesador

El comportamiento del procesador SimpleSem se rige por el siguiente esquema:

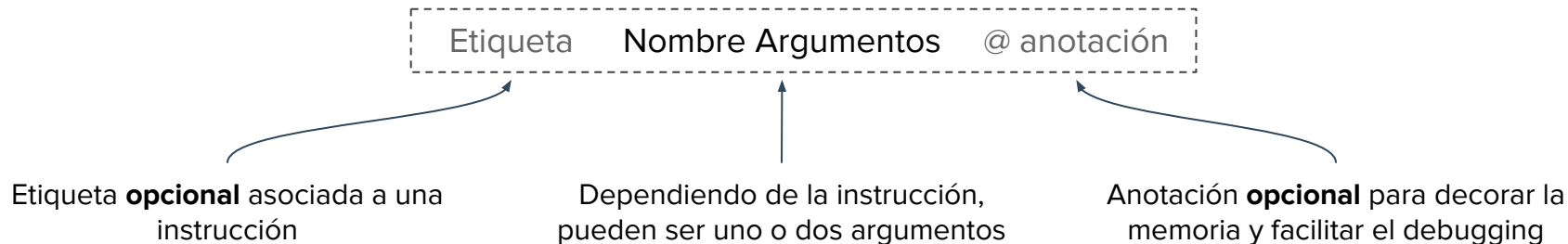
Mientras no encuentre instrucción **halt**:

Toma instrucción de código referenciada por el registro PC.

Ejecuta la instrucción.

Incrementa valor de PC en 1 (si la instrucción no lo modifica).

Una **instrucción** tiene la siguiente forma:



```
met1A SetH D[Actual+2]+1, D[Actual+3] @ x=p
```

Funcionamiento del Procesador

Algunas de las instrucciones de SimpleSem son las siguientes:

Instrucciones de modificación de memoria

SetD <Locación>, <Valor>

SetH <Locación>, <Valor>

Instrucciones de salto

Jump <Locación>

JumpT <Locación>, <Condición>

Instrucciones de modificación de registros:

SetActual <Valor>

SetLibre <Valor>

SetPO <Valor>

Instrucción para finalizar la ejecución

Halt

Cada argumento (<Locación>, <Valor>, <Condición>) puede ser una **expresión**.

Ver [manual](#) para el resto de instrucciones disponibles.

Funcionamiento del Procesador

Las expresiones en SimpleSem son **expresiones matemáticas y/o lógicas** que pueden involucrar accesos a memoria (D y H) o al valor de los registros (Actual, Libre, PC).

<Expresión>	<Expresión> <OpBinaria> <Expresión>
	<OpUnaria> <Expresión>
	<Operando>
<Operando>	D[<Expresión>]
	H[<Expresión>]
	Actual
	Libre
	PC
	Número
	Identificador
	(<Expresión>)

Algunos ejemplos de expresiones

1 + 2

7 == 6

!(7 == 6)

Actual + 1

D[Actual + 2] + 2

D[D[H[D[Actual+2]+0]+4]+0]

D[Actual+3] + D[Actual + 4]

Consultar el [manual](#) para ver la gramática completa.

Estructuras en Ejecución

Estructuras en Ejecución

Contamos con cuatro tipos de estructuras que nos permitirán resolver correctamente el flujo de la ejecución del programa:

1. **Class Instance Records (CIRs o INSTs)**

Almacenan información de las instancias, como el valor de los atributos de instancia y una referencia a la Virtual Table de la clase

2. **Class Records (CRs)**

Almacenan información de la clase, como son los atributos de clase

3. **Virtual Tables (VTs)**

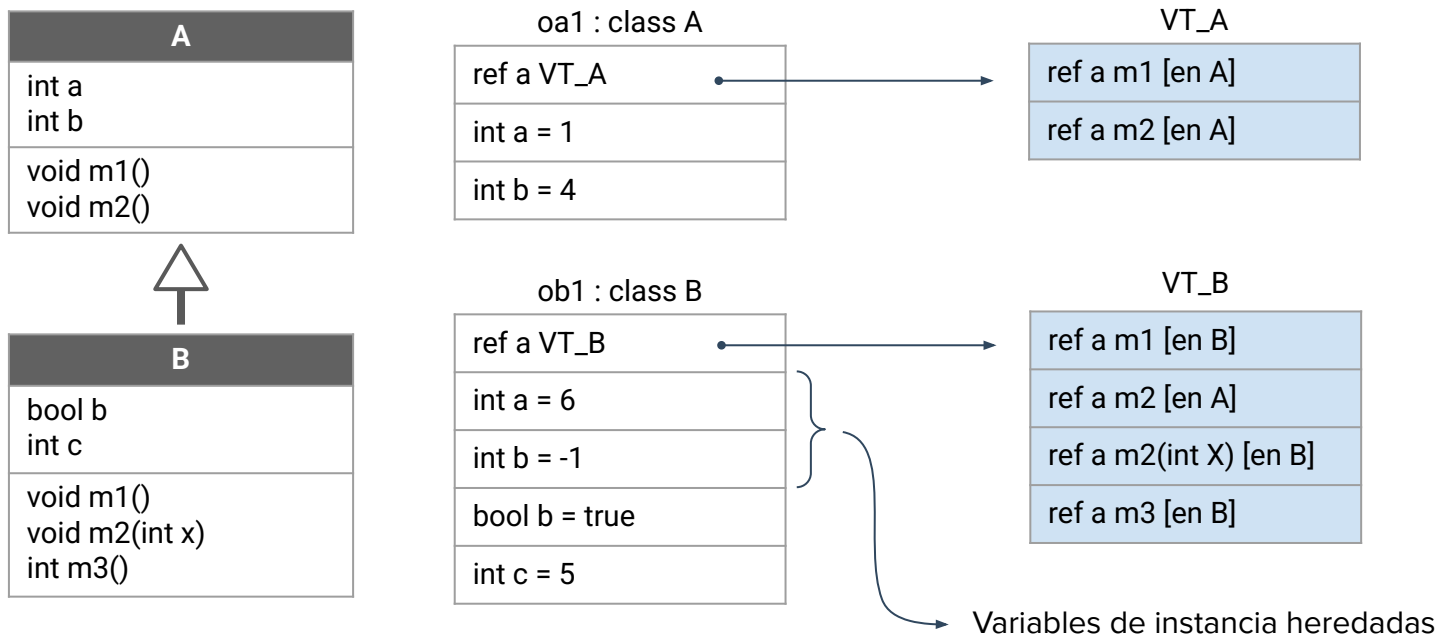
Almacenan referencias a los métodos vinculados a cada clase

4. **Registros de Activación (RA)**

Almacena los datos necesarios para la correcta ejecución de una unidad

Class Instance Record (INST)

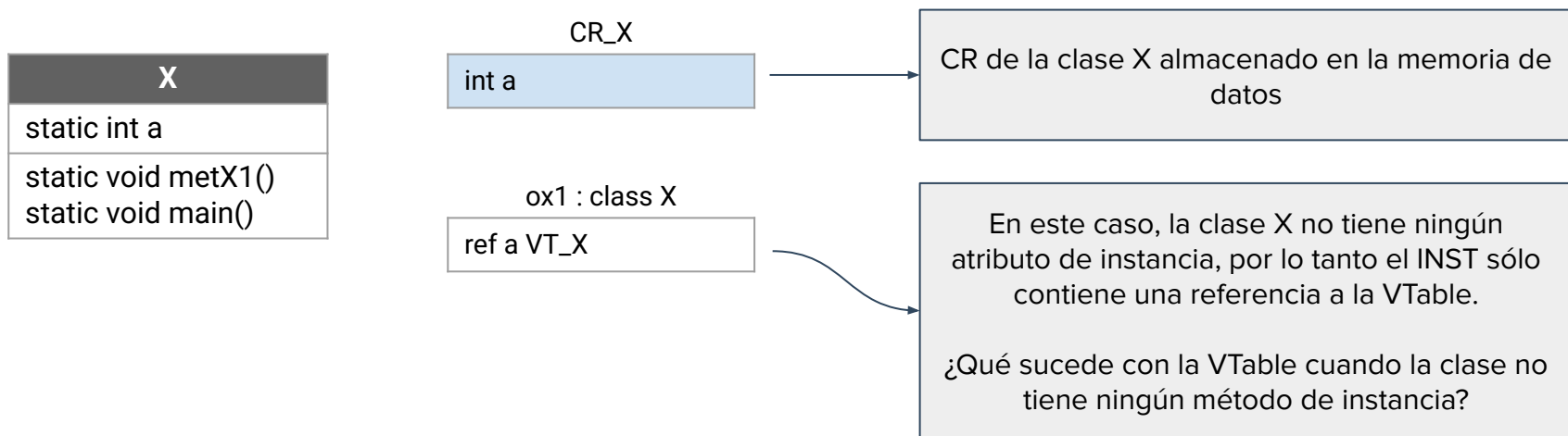
Los INST se crean de manera dinámica durante la ejecución del programa cada vez que se crea un nuevo objeto instancia de una clase, y **se almacenan en la memoria dinámica** (heap).



Class Record (CR)

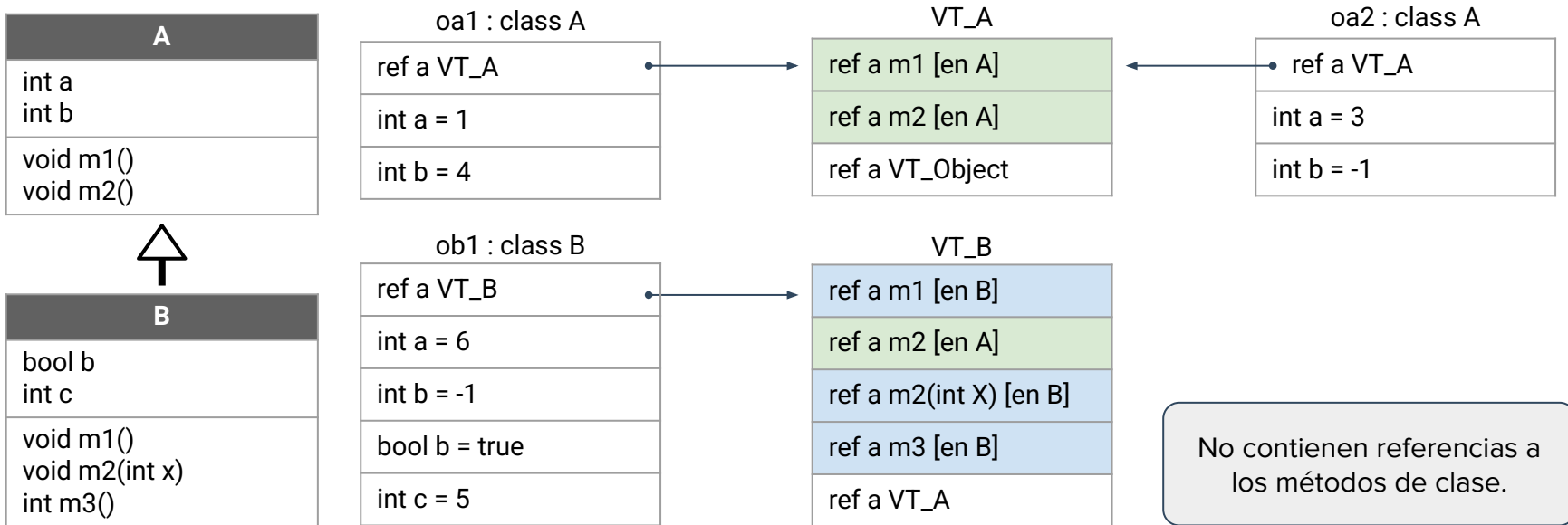
Los CRs almacenan **información referente a una clase**, en particular los atributos de clase. **Se almacenan en la memoria de datos**, ya que se crean en compilación o en ejecución, en un momento anterior a comenzar la ejecución del programa a través de su método “main”.

Tendremos un **único CR para cada clase**, que será compartido por todos los objetos instancia de esa clase.



Virtual Tables (VTables)

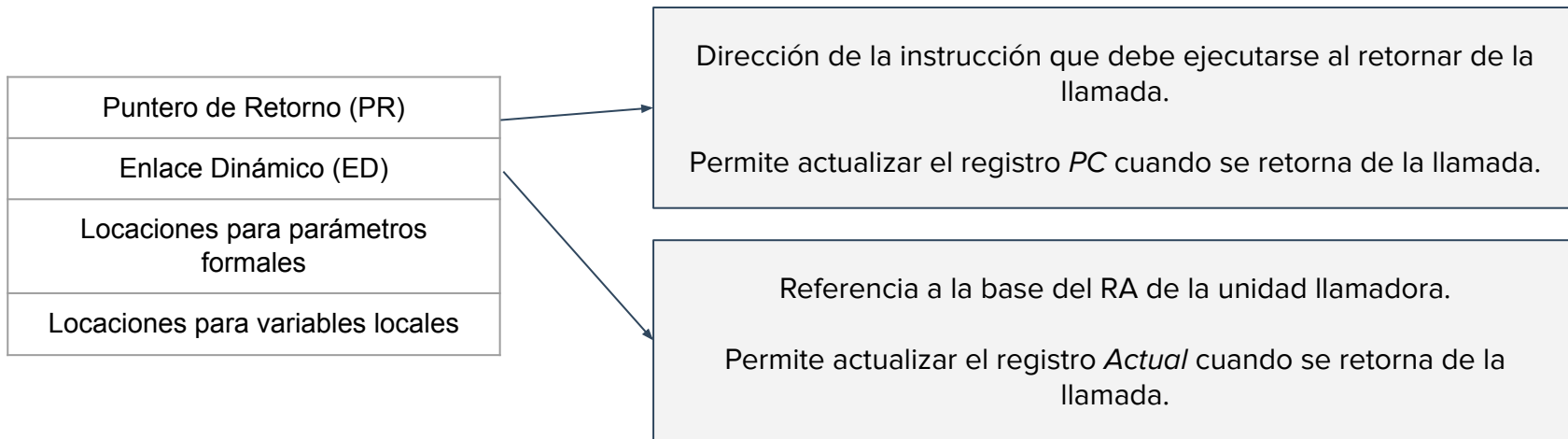
Las VTables almacenan información para **resolver la vinculación dinámica de código**. Tendremos **una única VTable para cada clase del programa**, que almacenará referencias al código de los métodos de instancia vinculados a dicha clase. Las VTables **se almacenan en la memoria de datos**, ya que se crean antes de comenzar la ejecución del programa. Esto puede ser en compilación o en ejecución.



Registro de Activación (RA)

A la hora de invocar un método vamos a tener que construir su registro de activación, en el cual debemos guardar toda la **información necesaria para la correcta ejecución de la unidad invocada**.

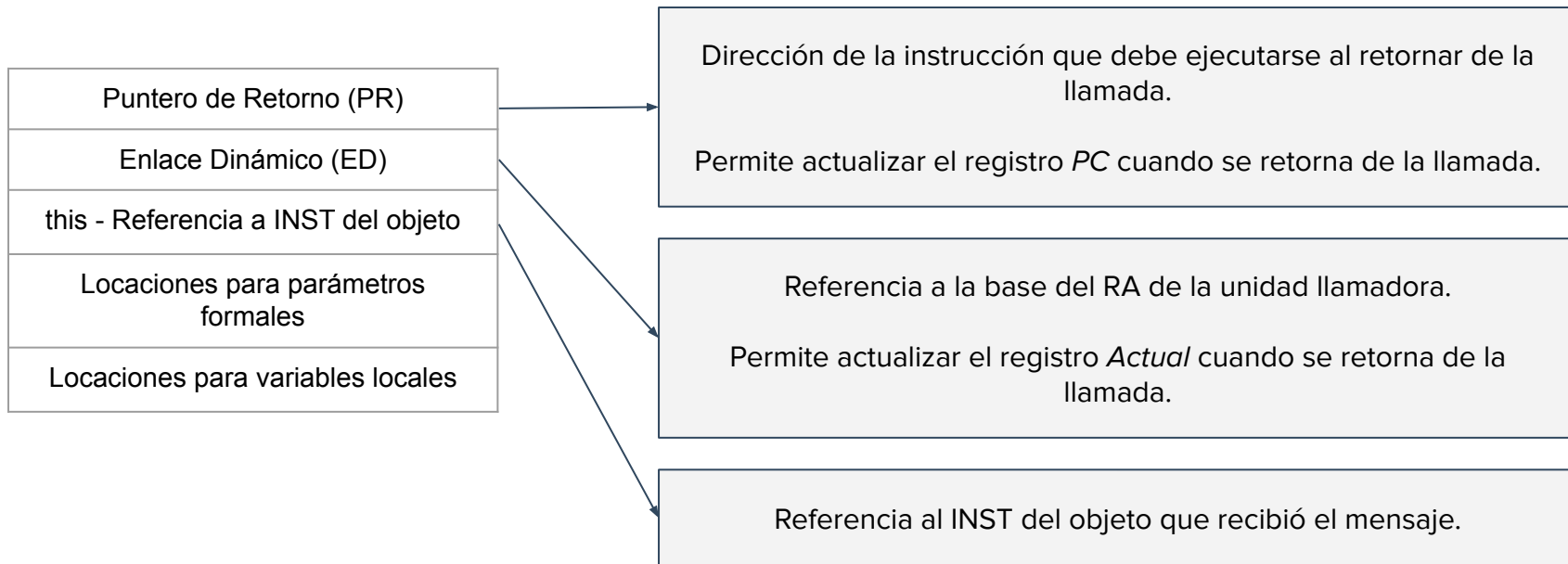
RA para métodos de clase (static)



Registro de Activación (RA)

A la hora de invocar un método vamos a tener que construir su registro de activación, en el cual debemos guardar toda la **información necesaria para la correcta ejecución de la unidad invocada**.

RA para métodos de instancia y constructores

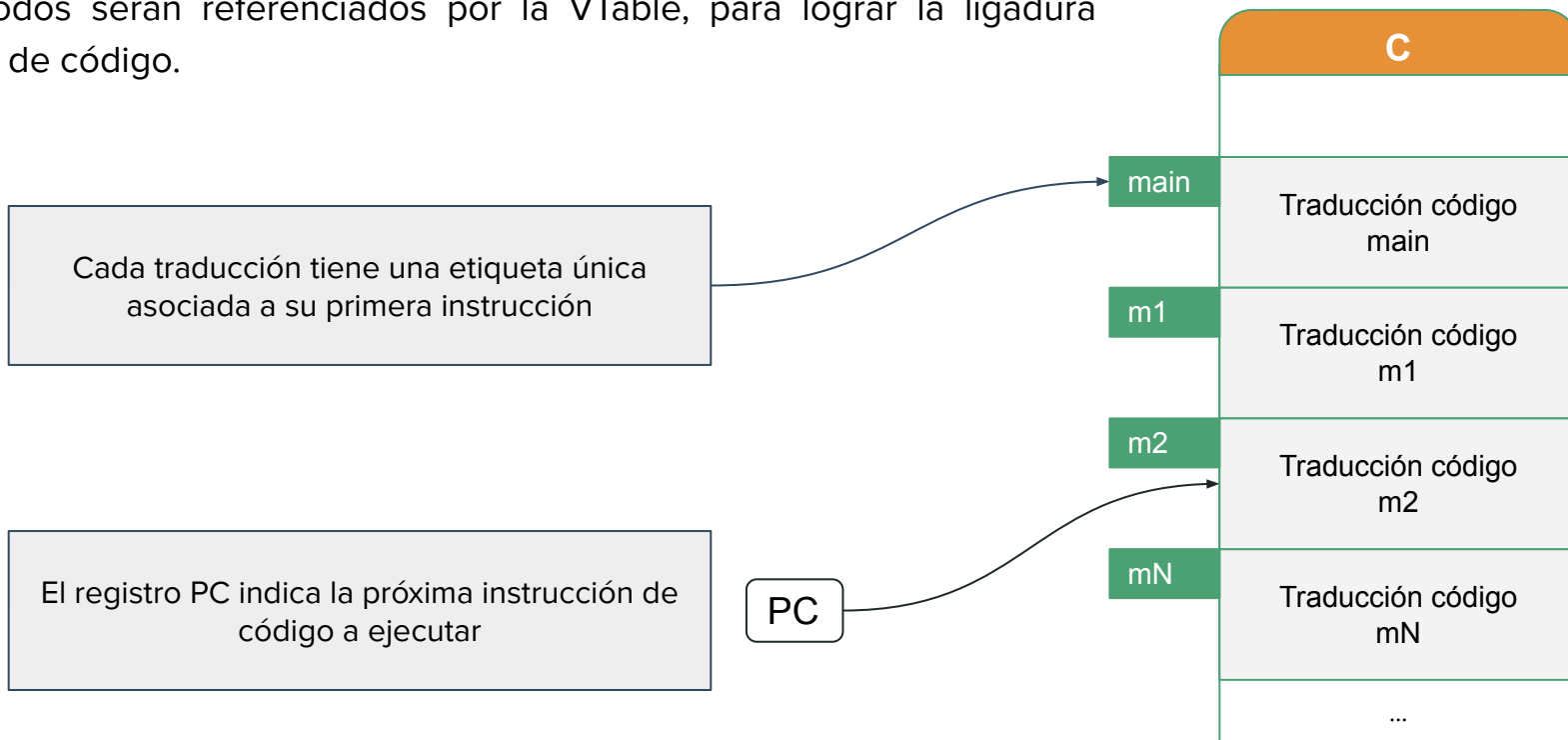


Memorias

Memoria de Código C

En la memoria de Código se **almacena la traducción de cada método** en el programa fuente.

Los métodos serán referenciados por la VTable, para lograr la ligadura dinámica de código.



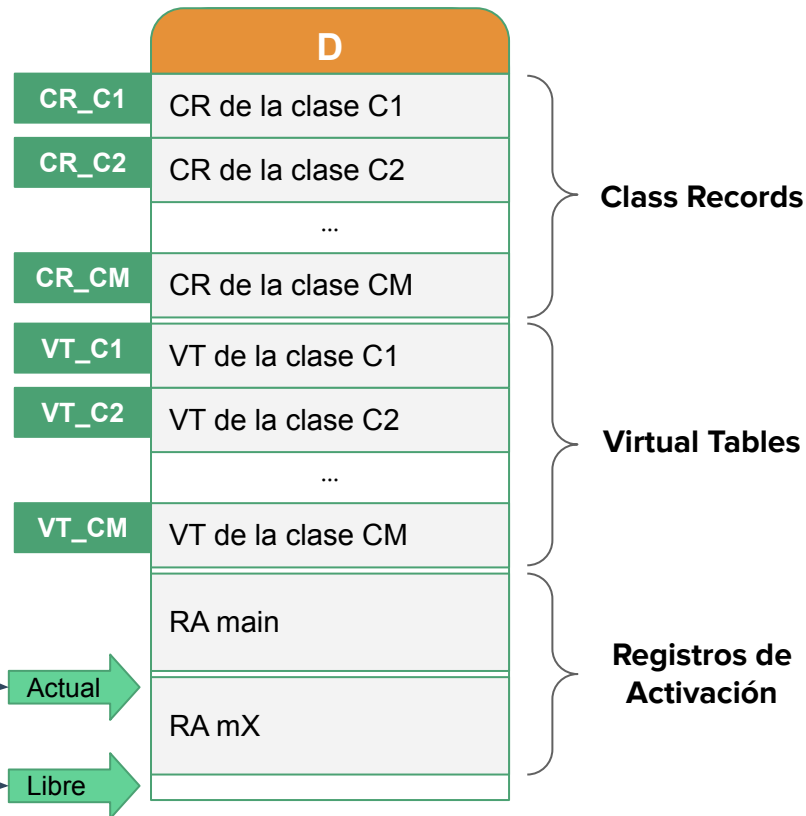
Memoria de Datos D

En la memoria de Datos **se almacenan los Registros de Clase, las VTables y los Registros de Activación** de las unidades invocadas.

Los registros de clase y las VTables tienen una etiqueta única asociada

El registro **Actual** marca el comienzo del registro de activación de la unidad actualmente en ejecución

El registro **Libre** marca la primera posición de memoria libre



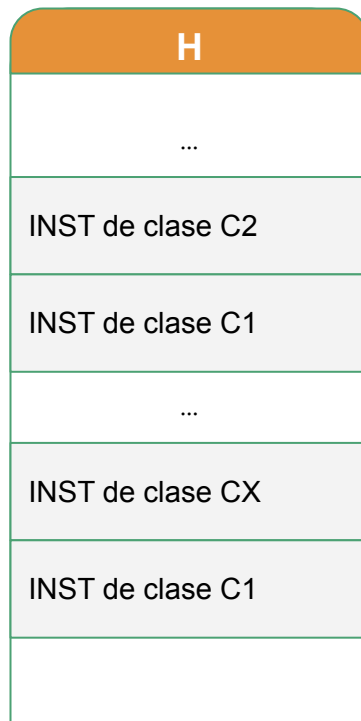
Memoria Dinámica o Heap H

En la memoria Heap **se almacenan los Registros de Instancia** de los objetos activos en ejecución. Todo INST va a tener una referencia a la base de la VTable de su clase.

El registro PO indica la primera posición de memoria libre. Nos permitirá ubicar un nuevo INST cuando se cree un objeto en ejecución

PO

En SimpleSem nos abstraemos de la gestión del Heap



Componentes del Procesador Abstracto

Se cuenta con **cuatro registros especiales** para manipular la memoria:

Registro Actual
Indica el comienzo del registro de activación de la unidad que está en ejecución en cada momento. Actúa en la memoria de datos D.
Registro Libre
Indica la primera posición de memoria libre en la memoria de datos. Actúa en la memoria de datos D.
Registro PC
Indica la próxima instrucción de código a ejecutar. Actúa en la memoria de código C.
Registro PO
Indica la primera posición libre en la memoria Heap. Actúa en la memoria dinámica H.

Invocación y Retorno de una Unidad

Invocación a una Unidad

Debemos tener presente qué tareas se deben realizar en el **proceso de invocación a una unidad**.

1. **Crear el RA** de la unidad invocada.
2. **Actualizar el registro Actual** para que apunte a la base del nuevo RA.
3. **Actualizar el registro Libre** para que apunte a la primera dirección libre luego del RA recién creado.
4. **Actualizar el registro PC** para que apunte a la primera instrucción de código del método invocado.

Recordar que la estructura del RA cambia ligeramente dependiendo de si la unidad invocada es estática o dinámica.

Invocación a una Unidad Estática

Si la unidad invocada es estática, **no se guarda la referencia al objeto receptor** (this). Los pasos involucrados son:

1	Reservar espacio para el valor de retorno (si es necesario)	SET Libre, Libre + 1	}	Creación del RA
2	Guardar PR	SETD Libre, PC + Cant. Instrucciones de la Invocación		
3	Guardar ED	SETD Libre+1, Actual		
4	Parámetros (si es necesario)	SETD Libre + Lugar Param., Valor		
5	Actualizar Actual	SET Actual, Libre		
6	Actualizar Libre	SET Libre, Libre + Tamaño RA		
7	Actualizar PC	JUMP Dir. del método		

Invocación a una Unidad Dinámica

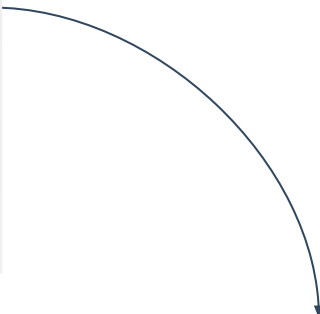
Si la unidad invocada es dinámica, el RA debe **incluir una referencia al objeto receptor del mensaje** (this). Los pasos involucrados entonces son:

1	Reservar espacio para el valor de retorno (si es necesario)	SET Libre, Libre + 1	}	Creación del RA
2	Guardar PR	SETD Libre, PC + Cant. Instrucciones de la Invocación		
3	Guardar ED	SETD Libre+1, Actual		
4	Guardar <i>this</i>	SETD Libre+2, Ref. Objeto Receptor		
5	Parámetros (si es necesario)	SETD Libre + Lugar Param., Valor		
6	Actualizar Actual	SET Actual, Libre		
7	Actualizar Libre	SET Libre, Libre + Tamaño RA		
8	Actualizar PC	JUMP Dir. del método en la VT del receptor		

Retorno de una Unidad

La etapa de retorno de la unidad invocada, sea dinámica o estática, involucra los siguientes pasos:

1	Almacenar retorno (si es necesario)	SETD Actual - 1, Valor de Retorno
2	Actualizar Libre	SET Libre, Actual
3	Actualizar Actual	SET Actual, D [Libre + 1]
4	Actualizar PC	JUMP D[Libre]



Guardamos el valor de retorno en la celda de memoria que habíamos dejado para este propósito en la invocación

Acceso a Datos

Acceso a Datos

Los datos a los que vamos a acceder durante la ejecución del programa pueden ser:

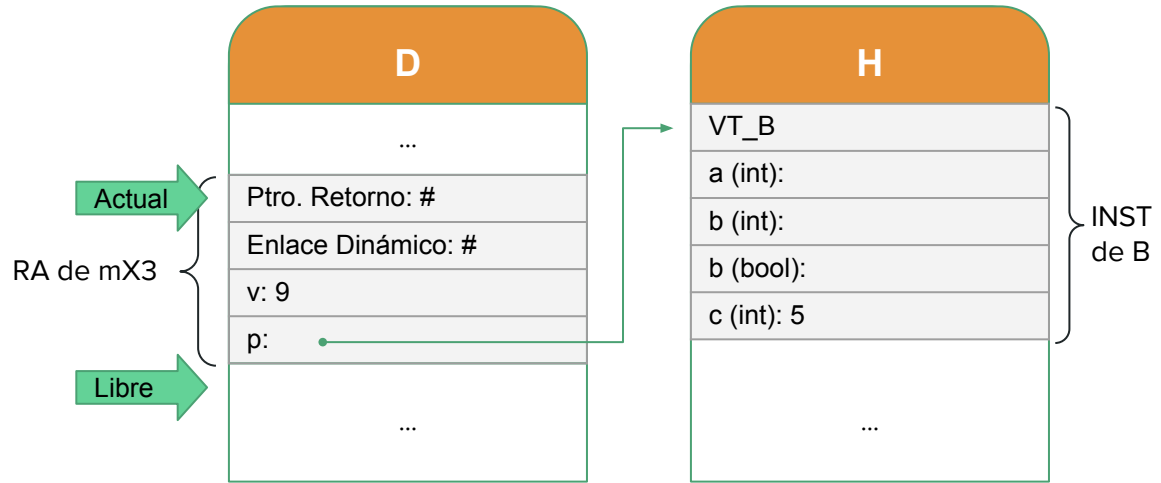
- Variables locales y parámetros en un RA.
- Variables de instancia, en un INST.
- Variables de clase, en un CR.
- Direcciones de métodos, en las VTables de las clases.

En general, vamos a acceder a estos datos mediante una referencia a la base del RA, INST, CR o VTable, sumándole el offset del dato deseado dentro de la estructura.

Acceso a Datos :: Acceso en un RA

En este caso, veamos cómo acceder al valor de parámetro formal entero.

```
static void mX3(int v, B p){  
    v = v + 1;  
    p.c = p.c + 10;  
}
```



Acceso a la locación del parámetro v

Actual + 2

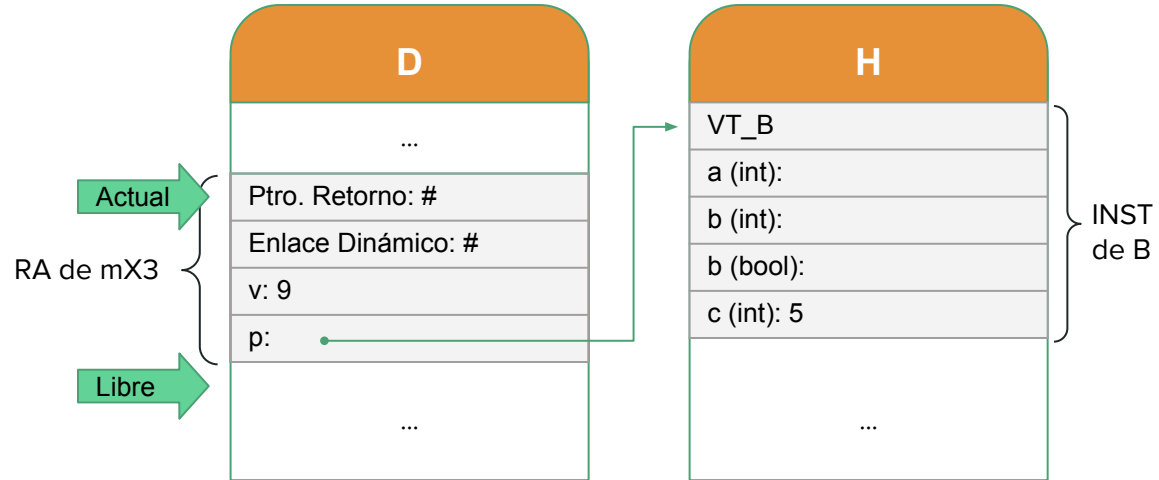
Acceso al valor del parámetro v

D[Actual + 2]

Acceso a Datos :: Acceso en un RA

En este caso, veamos cómo acceder a un atributo de instancia de un objeto recibido por parámetro.

```
static void mX3(int v, B p){  
    v = v + 1;  
    p.c = p.c + 10;  
}
```



Acceso a la locación del atributo c del parámetro p

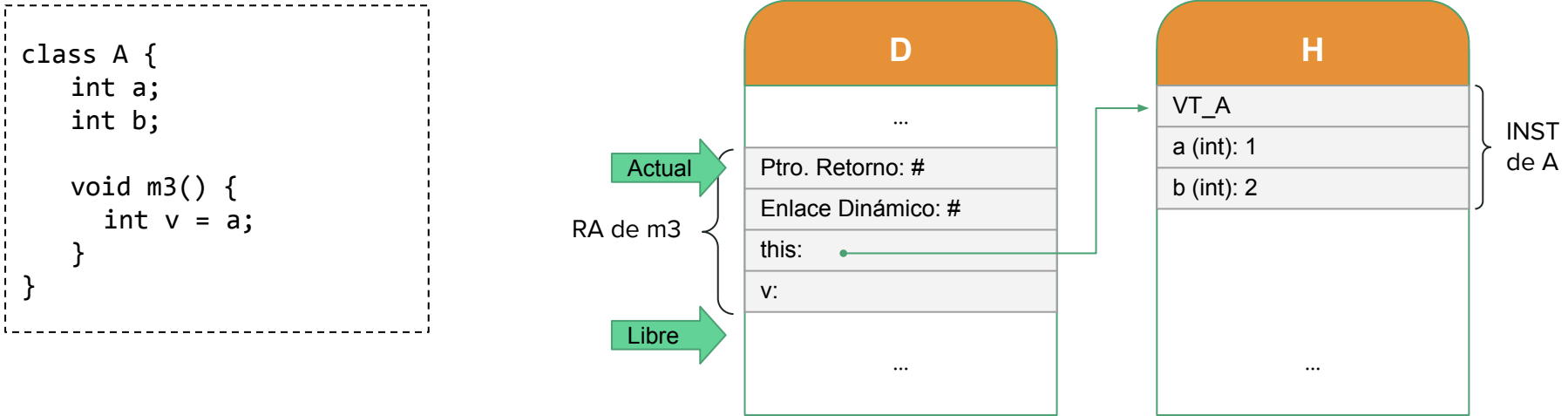
$D[\text{Actual} + 3] + 4$

Acceso al valor del atributo c del parámetro p

$H[D[\text{Actual} + 3] + 4]$

Acceso a Datos :: Acceso en un INST

Veamos ahora cómo acceder a un atributo de instancia del objeto receptor del mensaje.



Acceso a la locación del atributo de instancia a

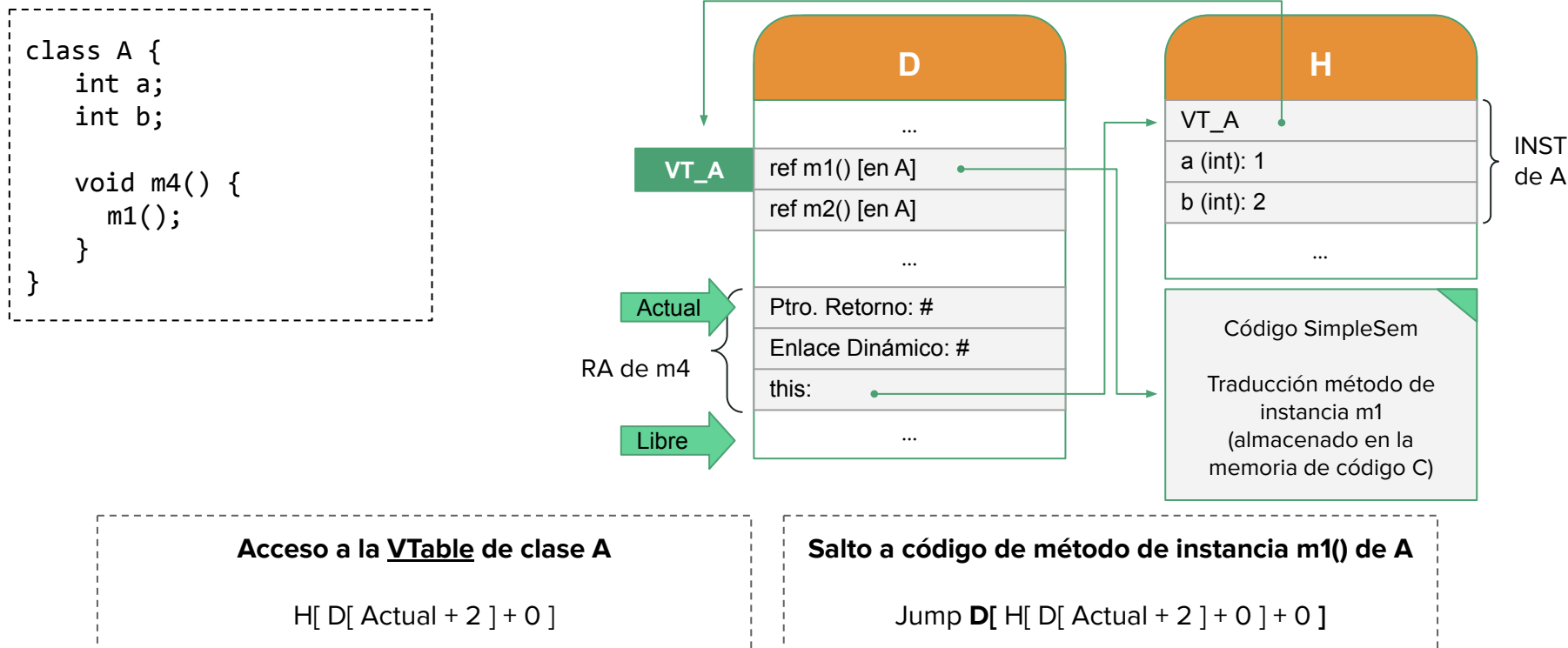
$D[\text{Actual} + 2] + 1$

Acceso al valor del atributo de instancia a

$H[D[\text{Actual} + 2] + 1]$

Acceso a Datos :: Acceso en una VTable

Finalmente, debemos poder acceder al código de un método ante una invocación.



Modificación de Datos

Modificación de Datos

Además de acceder a los datos, ya sea dentro de un RA, de un INST, de un CR o una VTable, también podremos modificar los valores asociados a variables.

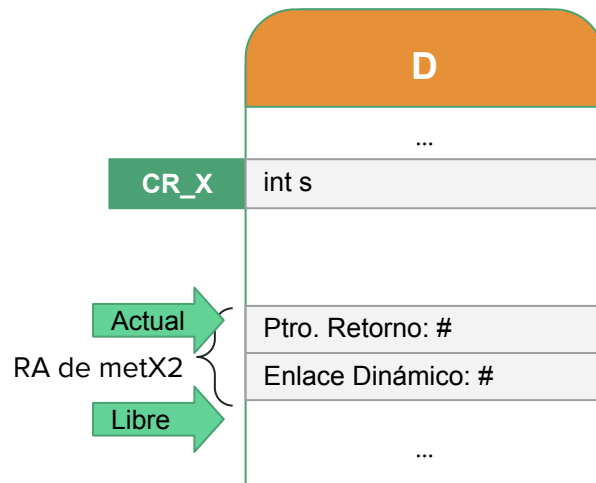
En particular, podríamos ligar un nuevo valor a:

- Un atributo de clase.
- Un atributo de instancia.
- Un parámetro formal.
- Un atributo de instancia de un parámetro formal.

Modificación de Datos :: Atributo de Clase

Podemos acceder y modificar un atributo de clase a partir de la etiqueta asociada al CR.

```
class X {  
    static int s;  
  
    static void metX2(){  
        s = s + 10;  
    }  
}
```



Acceso a la locación del atributo s

CR_X (+0)

Acceso al valor del atributo s

D[CR_X]

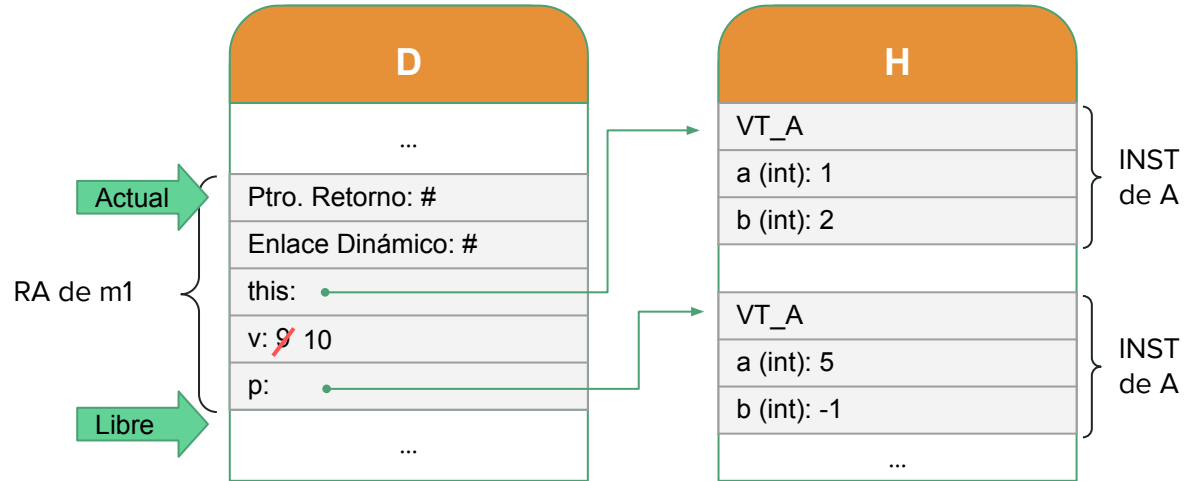
Seteo del valor del atributo s

SETD CR_X, D[CR_X] + 10

Modificación de Datos :: Parámetro

Veamos como modificar un parámetro de un método.

```
void m1(int v, A p){  
    v = v + 1;  
    a = v;  
    p.b = p.b + 10;  
}
```



Acceso a la locación del parámetro v

Actual + 3

Acceso al valor del parámetro v

D[Actual + 3]

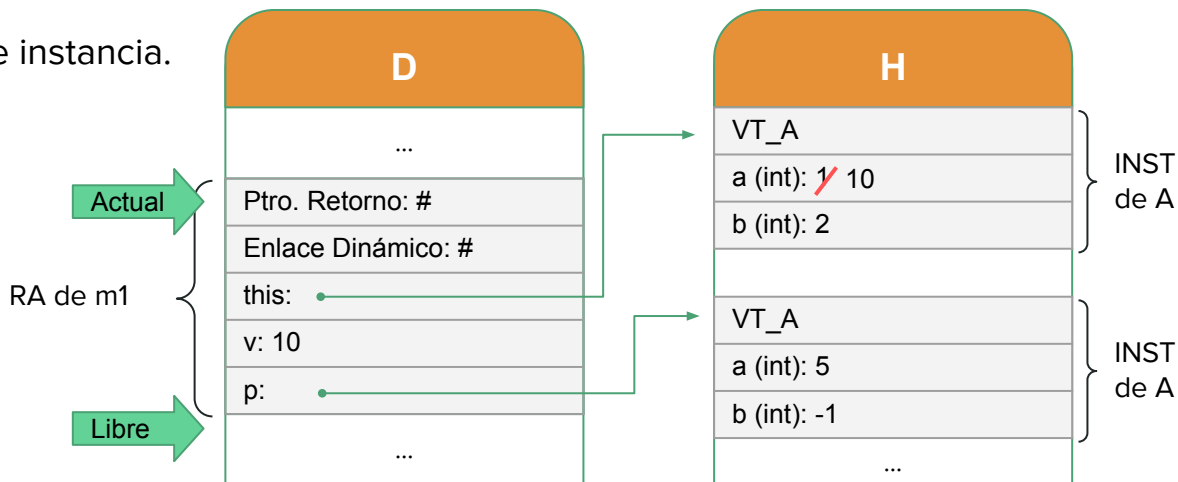
Seteo del valor del parámetro v

SETD Actual + 3, D[Actual + 3] + 1

Modificación de Datos :: Atributo de Instancia

Veamos como modificar un atributo de instancia.

```
void m1(int v, A p){  
    v = v + 1;  
    a = v;  
    p.b = p.b + 10;  
}
```



Acceso a la locación de la variable de instancia a

$D[\text{Actual} + 2] + 1$

Acceso al valor de la variable de instancia a

$H[D[\text{Actual} + 2] + 1]$

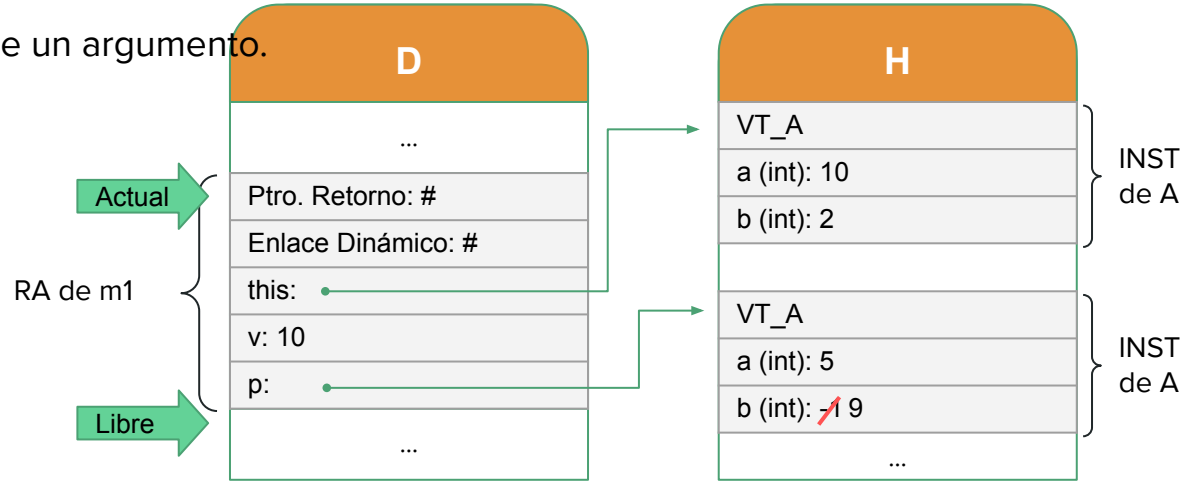
Seteo del valor de la variable de instancia a

SETH $D[\text{Actual} + 2] + 1, D[\text{Actual} + 3]$

Modificación de Datos :: Variable de Instancia de un Parámetro

Veamos como modificar un atributo de un argumento.

```
void m1(int v, A p){  
    v = v + 1;  
    a = v;  
    p.b = p.b + 10;  
}
```



Acceso a la locación del atributo b del parámetro p

$D[\text{Actual} + 4] + 2$

Acceso al valor del atributo b del parámetro p

$H[D[\text{Actual} + 4] + 2]$

Seteo del valor del atributo b del parámetro p

SETH $D[\text{Actual} + 4] + 2, H[D[\text{Actual} + 4] + 2] + 10$

Ejemplo Completo

Ejemplo

Vamos a realizar la traducción completa del siguiente código.

```
class A {  
    int a, b;  
  
    void m1() {  
        a = 1;  
        b = -1;  
    }  
  
    void m2() {  
        a = b + 1;  
        b = b + 5;  
    }  
}
```

```
class B extends A {  
    bool b;  
    int c;  
  
    void m1() {  
        super.m1();  
        c = m3();  
        b = true;  
    }  
    void m2(int x){  
        x = x + 1;  
        a = a + x;  
    }  
    int m3(){  
        m2(1);  
        return a;  
    }  
}
```

```
class X {  
    static int s;  
  
    static void metX1(A p){  
        p.m1();  
    }  
  
    static void main(...){  
        A v;  
        v = new B();  
        metX1(v);  
    }  
}
```

Ejemplo

% Creación CR y VTables

SetLabel CR_X, Libre	% etiqueta CR_X	}	% creación de CR_X
SetLibre Libre+ 1	% Actualizar Libre		
SetActual Libre	% Actualizar Actual		
SetLabel VT_A, Libre	% etiqueta VT_A	}	% creación de VT_A
SetD Libre, m1A	% dirección de m1 de A		
SetD Libre+1, m2A	% dirección de m2 de A		
SetLibre Libre+ 2	% Actualizar Libre		
SetActual Libre	% Actualizar Actual		

De manera similar creamos la demás VTables.

D	
	CRs...
CR_X	s:
	VTables...
VT_A	ref m1() [en A]
	ref m2() [en A]
VT_B	ref m1() [en B]
	ref m2() [en A]
	ref m2(int x) [en B]
	ref m3() [en B]

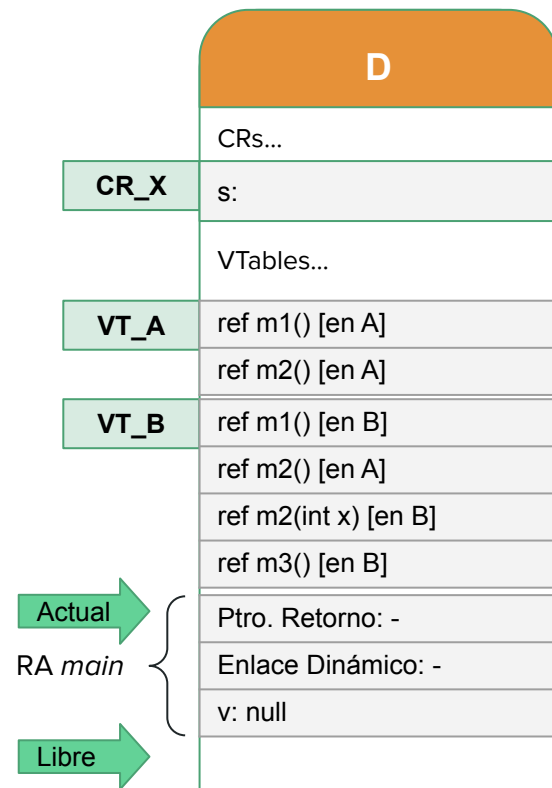
Ejemplo

% Invocación a main y halt

SetD Libre, PC + 5	% PR
SetD Libre + 1, Actual	% ED
SetActual Libre	% Actualizar Actual
SetLibre Actual + 3	% Actualizar Libre
Jump main	% Salto a código main
Halt	% Fin del programa

```
class X {  
    ...  
    static void main(...){  
        A v;  
        v = new B();  
        metX1(v);  
    }  
}
```

La dirección del código de los métodos estáticos se conoce siempre en tiempo de compilación, no hay vinculación dinámica (por lo tanto no se busca en la VTable).



En este caso, podemos omitir el puntero de retorno y el enlace dinámico.

Ejemplo

% Código método main

SetD Actual + 2, PO

SetH PO, VT_B

SetPO PO + 5

*% creación
objeto de
clase B*

SetD Libre, PC + 5

% PR

SetD Libre + 1, Actual

% ED

SetD Libre + 2, D[Actual+2]

% Parámetro p

SetActual Libre

% Actualizar Actual

SetLibre Actual + 3

% Actualizar Libre

Jump metX1

% Salto a código de metX1

SetLibre Actual

SetActual D[Libre + 1]

% Retorno de main

Jump D[Libre]

```
class X {  
    ...  
    static void main(...){  
        A v;  
        v = new B();  
        metX1(v);  
    }  
}
```

VT_B

D

CRs y VTs ...

ref m1() [en B]

ref m2() [en A]

ref m2(int x) [en B]

ref m3() [en B]

ref a VT_A

Ptro. Retorno: -

Enlace Dinámico: -

v:

Ptro. Retorno: -

Enlace Dinámico: -

p:

H

VT_B

int a:

int b:

bool b:

int c:

Actual

RA main

Libre

Actual

RA metX1

Libre

Ejemplo

% Código método metX1

SetD Libre, PC + **6**

SetD Libre + 1, Actual

SetD Libre+2, D[Actual + 2]

SetActual Libre

SetLibre Actual + **3**

Jump D[H[D[Actual+2]+0]+0]

SetLibre Actual

SetActual D[Libre + 1]

Jump D[Libre]

% PR

% ED

% this

% Actualizar Actual

% Actualizar Libre

% salto a código de m1 de B

% retorno de metX1

```
class X {  
    ...  
    static void metX1(A p){  
        p.m1();  
    }  
}
```

VT_B

D

H

CRs y VTs ...

ref m1() [en B]

ref m2() [en A]

ref m2(int x) [en B]

ref m3() [en B]

ref a VT_A

Ptro. Retorno: -

Enlace Dinámico: -

v:

Ptro. Retorno: -

Enlace Dinámico: -

p:

Ptro. Retorno: -

Enlace Dinámico: -

this:

VT_B

int a:

int b:

bool b:

int c:

RA main

RA metX1

RA m1 de B

Actual

Actual

Libre

Ejemplo

% Código método m1 de B

SetD Libre, PC + **6**

% PR

SetD Libre + 1, Actual

% ED

SetD Libre+2, D[Actual + 2]

% this

SetActual Libre

% Actualizar Actual

SetLibre Actual + **3**

% Actualizar Libre

Jump D[D[H[D[Actual+2]+0]+4]+0] % salto a código de m1 de A

Acceso a VT de clase A (padre) a través de referencia en la VTable de B.

```
class B extends A {  
    ...  
    void m1() {  
        super.m1() ;  
        c = m3();  
        b = true;  
    }  
}
```

VT_B

Actual

RA m1 de B

Libre

Actual

RA m1 de A

Libre

D

CRs y VTs ...

ref m1() [en B]

ref m2() [en A]

ref m2(int x) [en B]

ref m3() [en B]

ref a VT_A

...

Ptro. Retorno: -

Enlace Dinámico: -

this: .

Ptro. Retorno: -

Enlace Dinámico: -

this: .

H

VT_B

int a:

int b:

bool b:

int c:

Continuará...

Ejemplo

% Código método m1 de A

SetH D[Actual + 2] + 1, 1 % a = 1

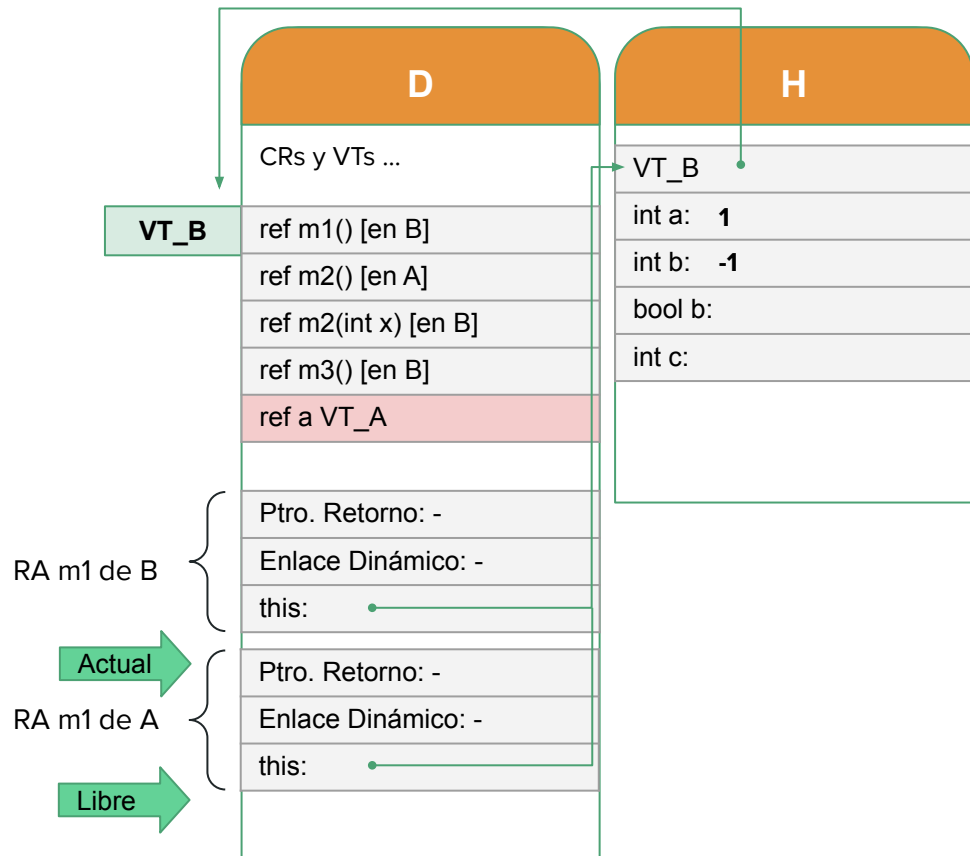
SetH D[Actual + 2] + 2, -1 % b = -1

SetLibre Actual

SetActual D[Libre + 1] % retorno de m1 de A

Jump D[Libre]

```
class B extends A {  
    ...  
    void m1() {  
        a = 1;  
        b = -1;  
    }  
}
```



Ejemplo

% **Continuación** Código método m1 de B

SetLibre Libre+1 % *Lugar retorno*

SetD Libre, PC + 6 % *PR*

SetD Libre + 1, Actual % *ED*

SetD Libre+2, D[Actual + 2] % *this*

SetActual Libre % *Actualizar Actual*

SetLibre Actual + 3 % *Actualizar Libre*

Jump D[H[D[Actual+2]+0]+3] % *salto a código de m3 de B*

```
class B extends A {  
    ...  
    void m1() {  
        super.m1();  
        c = m3();  
        b = true;  
    }  
}
```

VT_B

D

CRs y VTs ...

ref m1() [en B]

ref m2() [en A]

ref m2(int x) [en B]

ref m3() [en B]

ref a VT_A

...

Ptro. Retorno: -

Enlace Dinámico: -

this: .

Lugar Retorno m3

Ptro. Retorno: -

Enlace Dinámico: -

this: .

H

VT_B

int a: 1

int b: -1

bool b:

int c:

Actual

RA m1 de B

Libre

Actual

RA m3 de B

Libre

Continuará...

Ejemplo

% Código método m3 de B

SetD Libre, PC + 7 % PR

SetD Libre + 1, Actual % ED

SetD Libre+2, D[Actual + 2] % this

SetD Libre + 3, 1 % Valor parámetro actual

SetActual Libre % Actualizar Actual

SetLibre Actual + 4 % Actualizar Libre

Jump D[H[D[Actual+2]+0]+2] % salto a código de m2(x) de B

```
class B extends A {  
    ...  
    int m3(){  
        m2(1);  
        return a;  
    }  
}
```

VT_B

Actual

RA m3 de B

Libre
Actual

RA m2(x) de B

Libre

D

H

D
CRs y VTs ...
ref m1() [en B]
ref m2() [en A]
ref m2(int x) [en B]
ref m3() [en B]
ref a VT_A
...
Lugar Retorno m3
Ptro. Retorno: -
Enlace Dinámico: -
this: .
Ptro. Retorno: -
Enlace Dinámico: -
this: .
x: 1

H
VT_B
int a: 1
int b: -1
bool b:
int c:

Continuará...

Ejemplo

% Código método m2(int x) de B

SetD Actual + 3, D[Actual + 3] + 1 % $x = x + 1$

SetH D[Actual + 2] + 1,
H[D[Actual + 2] + 1] + D[Actual + 3] % $a = a + x$

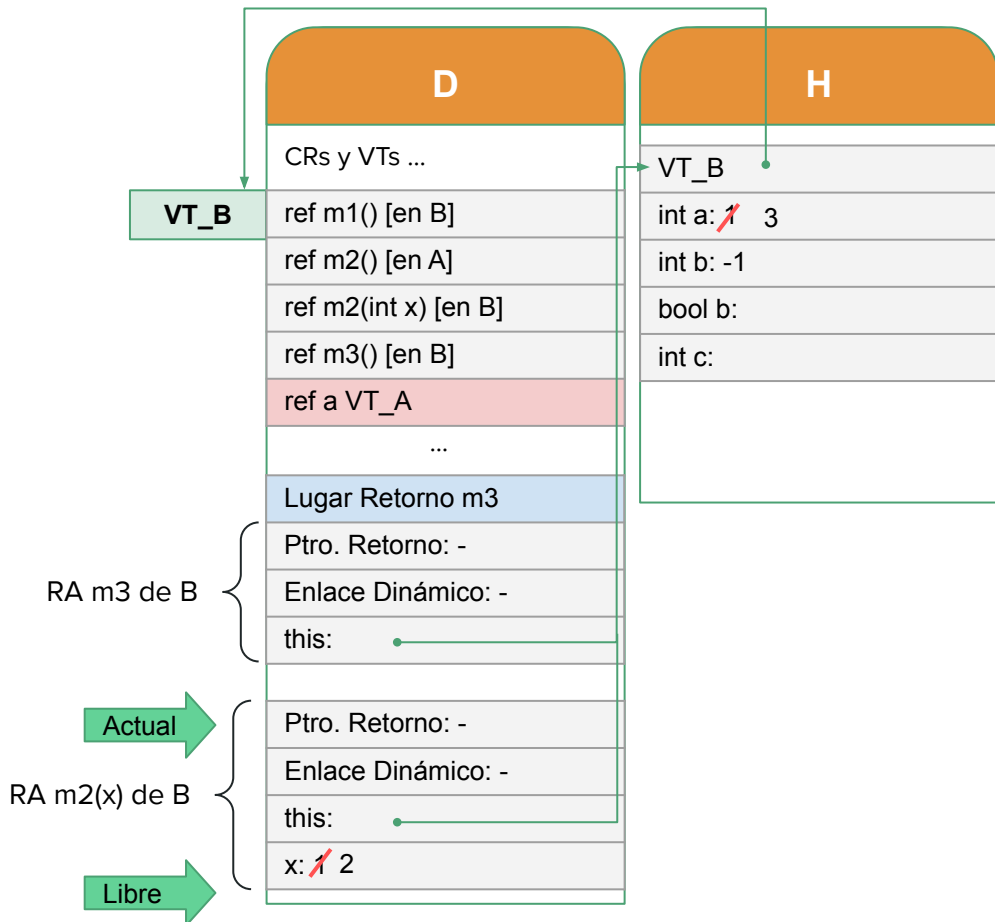
SetLibre Actual

SetActual D[Libre + 1]

Jump D[Libre]

% retorno de m2

```
class B extends A {  
    ...  
    void m2(int x){  
        x = x + 1;  
        a = a + x;  
    }  
}
```



Ejemplo

% Continuación Código método m3 de B

... habíamos llegado hasta el jump a m2(x) de B ...

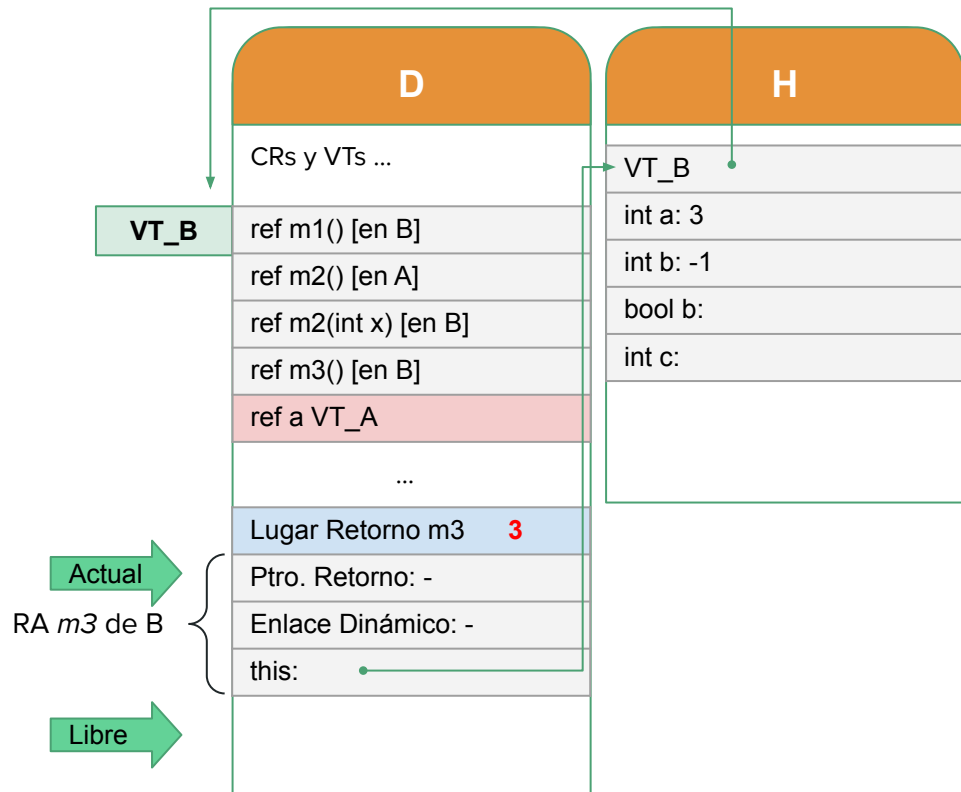
SetD Actual - 1, H[D[Actual + 2] + 1] % return a

SetLibre Actual

SetActual D[Libre + 1] % retorno de m3

Jump D[Libre]

```
class B extends A {  
    ...  
    int m3(){  
        m2(1);  
        return a;  
    }  
}
```



Ejemplo

% Continuación Código método m1 de B

... habíamos llegado hasta el jump a m3 de B ...

SetH D[Actual+2] +4, D[Libre - 1] % valor a c por x = m3()

SetH D[Actual+2] +3, true % b = true

SetLibre Libre - 1 % Quitamos lugar retorno

SetLibre Actual

SetActual D[Libre + 1]

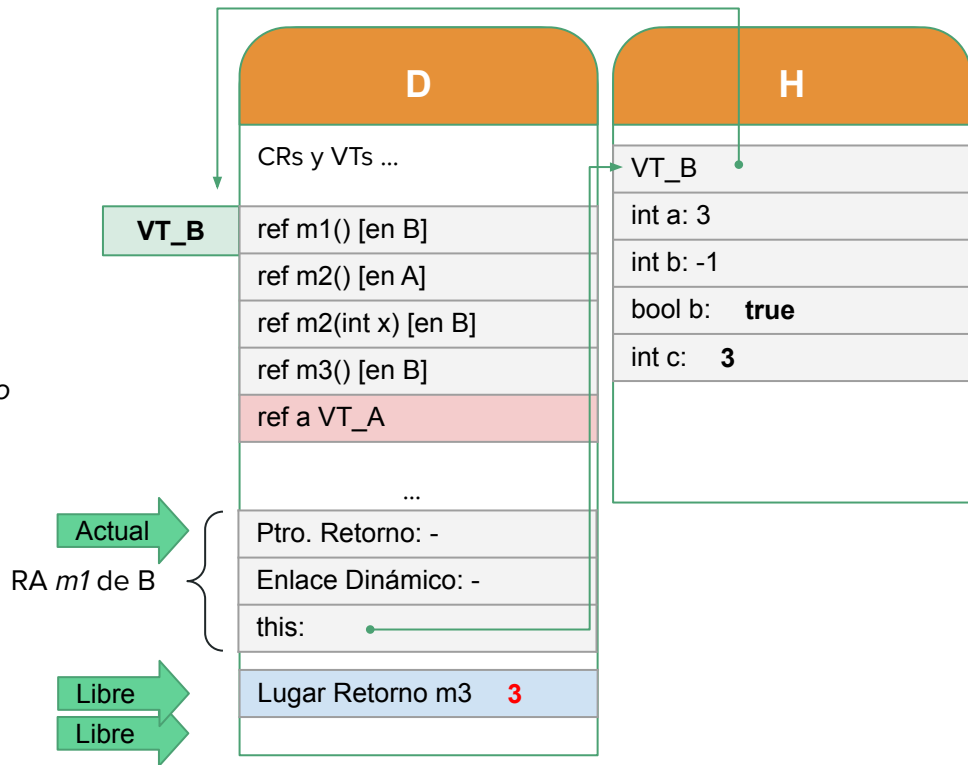
Jump D[Libre]

```
class B extends A {
```

```
...  
void m1() {  
    super.m1();  
    c = m3();  
    b = true;  
}
```

```
}
```

} % retorno de m1 de B



Recordar que en la pila aún tenemos los RA de main y metX1

Ejemplo

A modo de resumen, en el ejemplo anterior hemos visto cómo traducir a SimpleSem el programa, y en el proceso consideramos los siguientes aspectos:

- Dos clases en una jerarquía de herencia, con una clase cliente.
- Invocación a métodos de clase y métodos de instancia.
- Invocación a un método definido en la clase padre, usando *super*.
- Acceso y modificación de atributos de instancia de un objeto.
- Acceso y modificación del valor para un parámetro de un método.
- Acceso a la VTable para que el objeto pueda responder adecuadamente a los mensajes.

¿Preguntas?
